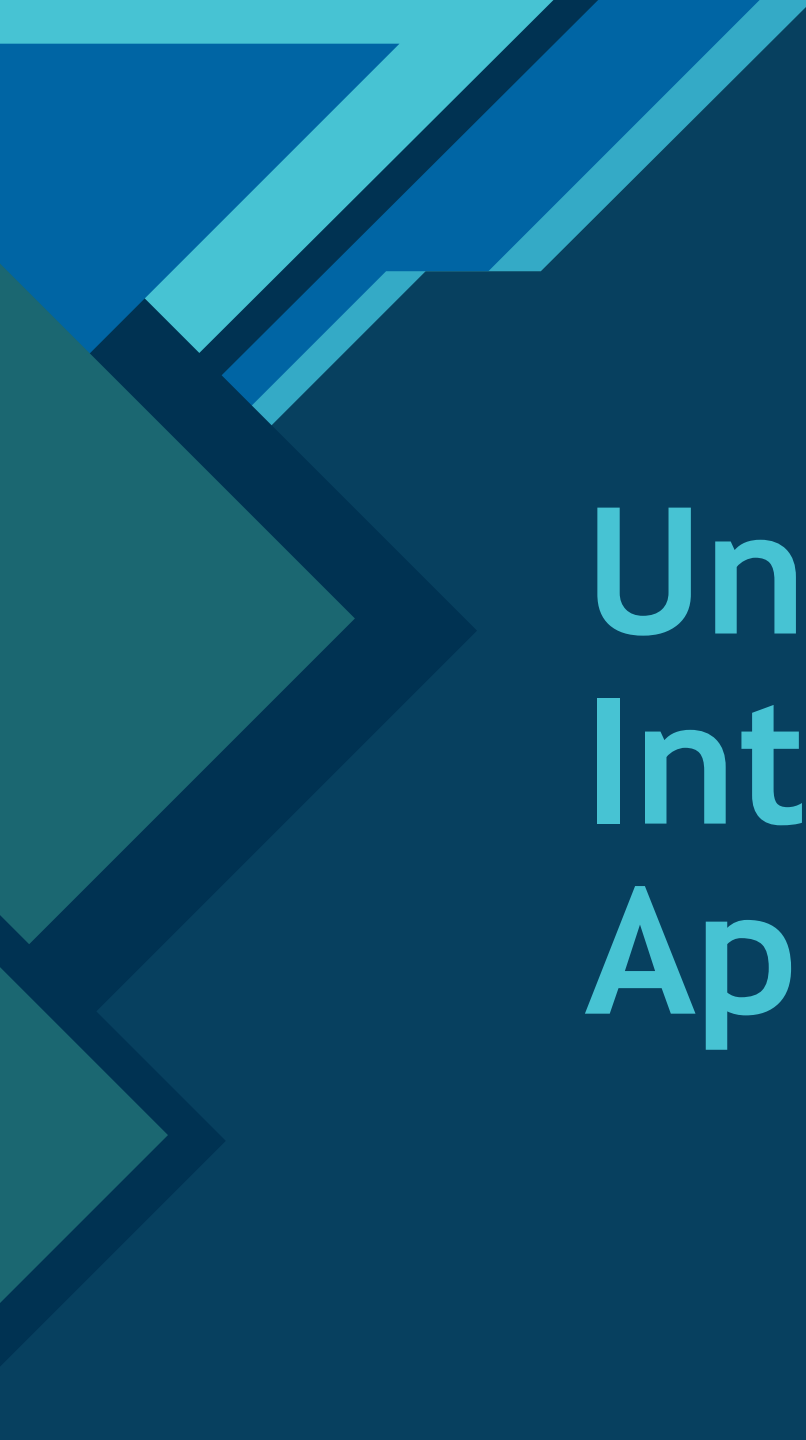




Unit 1: Introduction of Web Application

Unit 1: Introduction (6 hrs)

1.1 Overview of web applications and evolution of web architecture

1.2 Client-server architecture, HTTP, HTTPS, URLs, DNS, web browsers

1.3 Html basics: Syntax, tags, attributes, forms and inputs, tables, lists, multimedia, semantic HTML5 elements

1.4 CSS basics: Selectors, properties, values, box model

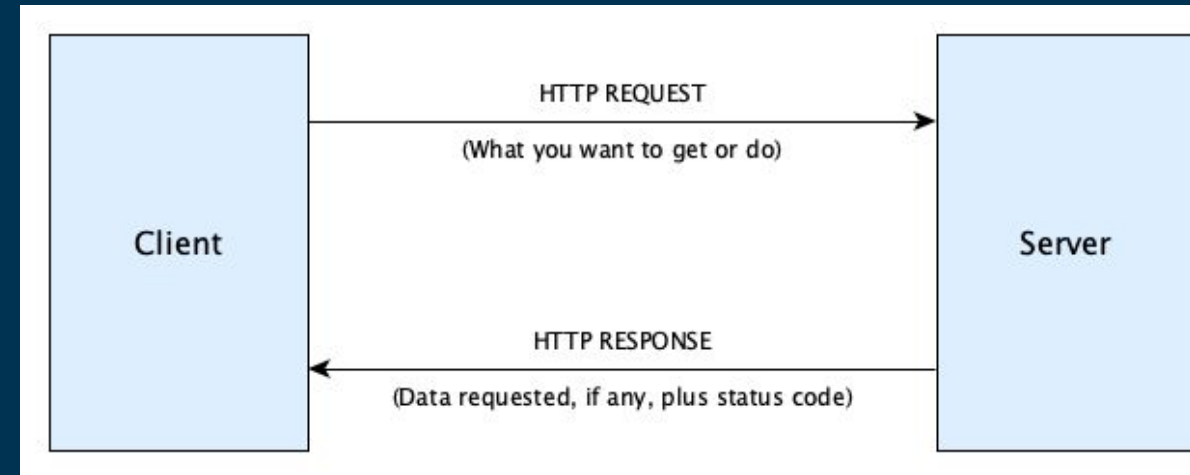
1.5 CSS framework: Bootstrap

1.1 Overview of web applications

- A web application is a software application accessed by users over a network, typically the internet, through a web browser.
- Loosely speaking, a web application is a program that runs on a computer with a web server and is accessed via a web browser (the “client”) that communicates with the server via the HTTP or HTTPS protocol.
- Unlike traditional desktop applications installed directly on a computer, web applications require no installation and can be accessed from various devices with internet connectivity.
- They are built using web technologies like HTML, CSS, and JavaScript, and often rely on server-side programming languages and databases for their functionality.

1.1 Overview of web applications

- In HTTP(S), the client sends a **request** to the server and the server replies with a **response** as shown in the figure aside.
- If the server always sends back the same data in the form of HTML, CSS, images, videos, and similar resources without any user interaction, we call the app a web site.
- If there is user interaction (mouse clicks, tapping and swiping, phone shaking, keypresses, etc.) that affect the execution of the program, we have a web app.
- If the server only produces raw data (generally in text or JSON), then we speak of a web service.



1.1 Overview of web applications

- Example:

- Gmail is a web app. All we need to use it is a web browser.
- We login, create and organize filters, read messages, reply, forward, send, and delete, and logout.
- Messages exist in a data store on the server, as does all the code to generate pages.
- Of course, the “pages” include a fair number of scripts that the browser knows how to execute, but these scripts are kept on the server and downloaded on demand.

1.1 Overview of web applications

- Writing and Deploying Simple Web Apps

- If our app is very simple, we can let our web browser take care of all of the details of the HTTP protocol, so we don't have to be concerned with it at all.
- We will just create (1) HTML to describe our app's content; (2) CSS to describe our app's presentation, and (3) JavaScript, to program the app's interactivity. We may also include (4) images, video, text, and other information.
- These resources are all placed on the server, and the clients will use a web browser to fetch and render them.
- Now we need to host our web app in web server. We can get our own server of course, or use one provided by a service.
- Services for simple web apps include firebase, DigitalOcean App Platform, AWS Amplify, Azure App Service, Glitch, and Code Sandbox.

1.1 Overview of web applications

- Key Characteristics and Benefits:

- No Installation Required: Users can access web apps instantly via a URL, eliminating the need for downloads or installations.
- Cross-Platform Compatibility: Web apps work seamlessly across various operating systems and devices, as long as a modern web browser is available.
- Automatic Updates: Updates are deployed on the server side, ensuring all users always have the latest version and security patches without manual intervention.
- Centralized Data and Security: Data is typically stored on the server, facilitating centralized management and enhanced security measures.
- Scalability: Web apps can easily accommodate a growing user base by upgrading server infrastructure.
- Interactive Functionality: They allow for dynamic interactions, enabling users to log in, submit data, receive real-time updates, and perform various tasks.

1.1 Overview of web applications

- How Web Applications Work:

Web applications function through a client-server architecture:

- Client-Side (Frontend): This is the part users interact with in their web browser, built using technologies like HTML, CSS, and JavaScript. It handles user interface, presentation, and client-side logic.
- Server-Side (Backend): This resides on a remote server and handles data storage, business logic, and communication with databases and other services. It is often built using languages like Python, Java, Node.js, or PHP.
- Communication: The client and server communicate using protocols HTTP/HTTPS requests and responses, allowing data exchange and dynamic content delivery.

1.1 Overview of web applications

- Examples of Web Applications:

Web applications are ubiquitous in daily digital life, including:

- Productivity Tools: Google Docs, Microsoft Office 365, Trello
- Social Media Platforms: Facebook, Twitter, Instagram
- E-commerce Sites: Amazon, eBay, online shopping carts
- Email Services: Gmail, Outlook Web App
- Content Management Systems (CMS): WordPress, Joomla
- Progressive Web Apps (PWAs): Offer app-like experiences with features like offline access and push notifications directly in the browser. Eg: twitter, Spotify, pintrest, AliExpress, MakeMyTrip

1.1 Evolution of web architecture

- Web application architecture guides the interactions between applications, middleware systems, user interfaces, and databases.
- All these components work together to create a robust, efficient, and engaging user experience on the web. As technology has evolved, so too have the architectures empowering these web applications.

1. The Birth of Web Applications: CGI and Early Dynamic Websites (1990s)

- In the early days of the web, static HTML pages ruled the internet. But as the internet grew in popularity, a push for more dynamic content emerged, leading to the creation of the Common Gateway Interface (CGI).
- With CGI, the web server could communicate with other programs on the server to generate dynamic, user-specific web content.

1.1 Evolution of web architecture (cont...)

2. Evolution Continued: From CGI to Multi-tiered Architectures (2000s)

- Multi-tier architectures started to become popular at the beginning of the millennium. The concept behind multi-tier architectures is separating concerns, where each layer of the application handles a specific task or tasks.
- The most common form, the three-tier architecture, usually consists of a presentation, application (business logic), and data tier. The separation of concerns among the tiers improves readability, scalability, and maintainability of the application.

3. Rise of MVC and MVVM Patterns (Late 2000s)

- By the late 2000s, web application requirements evolved significantly. There was a demand to create more robust, scalable, and complex applications that could handle heavy user interactions. This led to the rise of architectural patterns like Model-View-Controller (MVC) and Model-View-View-Model (MVVM).
- Both these patterns aimed to simplify development by decoupling data handling and user interface and enabling a more modular, maintainable structure. They were instrumental in the success of single-page applications (SPAs).

1.1 Evolution of web architecture (cont...)

4. Modern Age: Microservices and Serverless Architectures (2020s)

- The latest trend in web application architecture is breaking down application components into small, loosely coupled, and independently deployable services called microservices.
- These microservices can be up or down-scaled independently, making the architecture highly scalable and resilient.
- At the same time, serverless architectures, where server management is outsourced to cloud providers, have gained power. They allow developers to focus on writing code rather than managing servers, making the development process more efficient.

1.1 Evolution of web architecture (cont...)

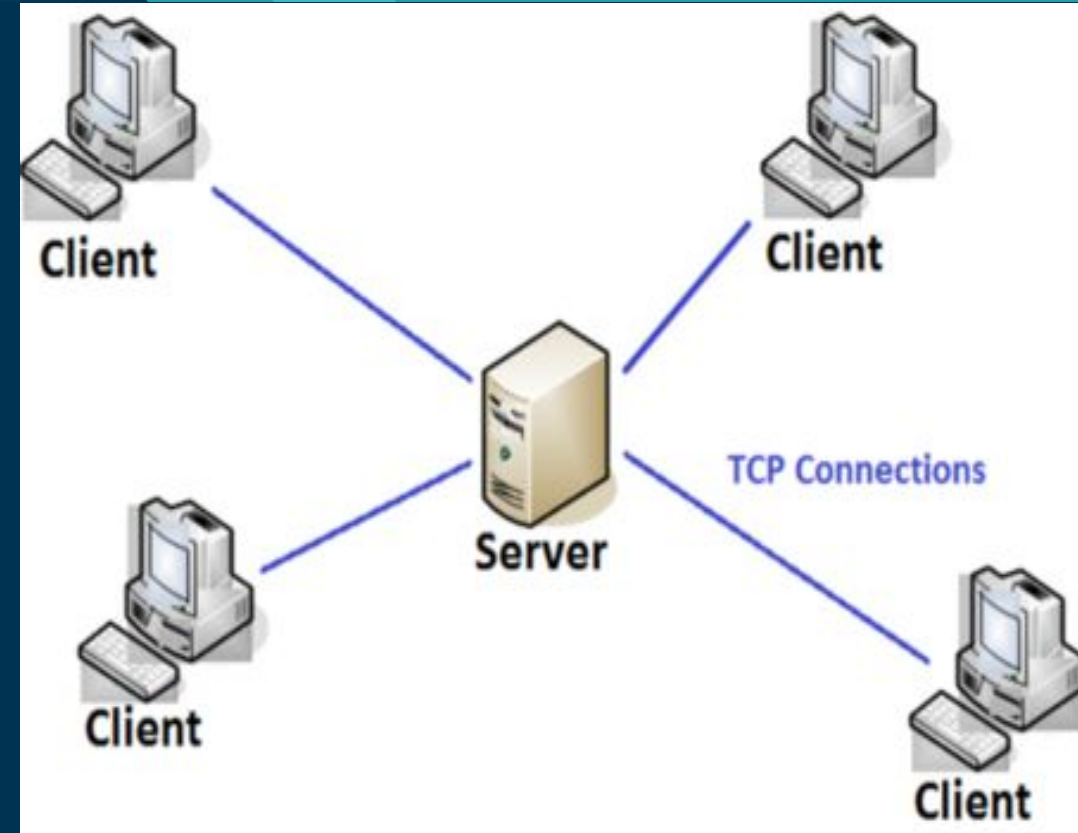
- Comparing the Architectures

While each architecture has its own merits, the choice typically depends on the specific use case:

- CGI had a simple architecture suitable for smaller, less complex sites.
- Multi-tier architectures offer a sensible way to separate concerns and works well for moderately complex applications.
- MVC and MVVM allow for a highly modular design that is perfect for complex, interactive web applications.
- Microservices and Serverless architectures, each with their own merits and demerits, fit in well with modern, cloud-first, high-performance applications that require extreme scalability and reliability.

1.2 Client Server Architecture

- Client-server architecture is an architecture of a computer network in which many clients (remote processors) request and receive service from a centralized server (host computer).
- In the client-server architecture, when the client computer sends a request for data to the server through the internet, the server accepts the requested, process it and deliver the data packets requested back to the client.
- One special feature is that the server computer has the potential to manage numerous clients at the same time.
- Also, a single client can connect to numerous servers at a single timestamp, where each server provides a different set of services to that specific client.

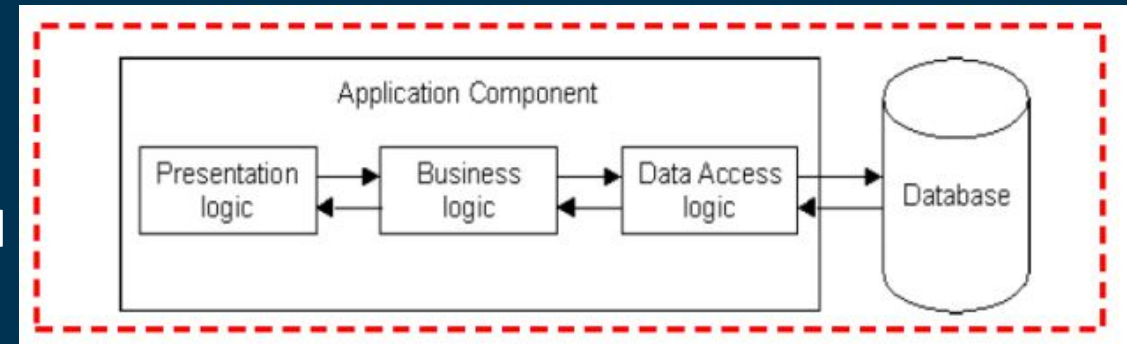


Client Server Architecture cont...

Client/Server architecture can be of different model based on the number of layers it holds. Some of them are:

1. Single tier Architecture:

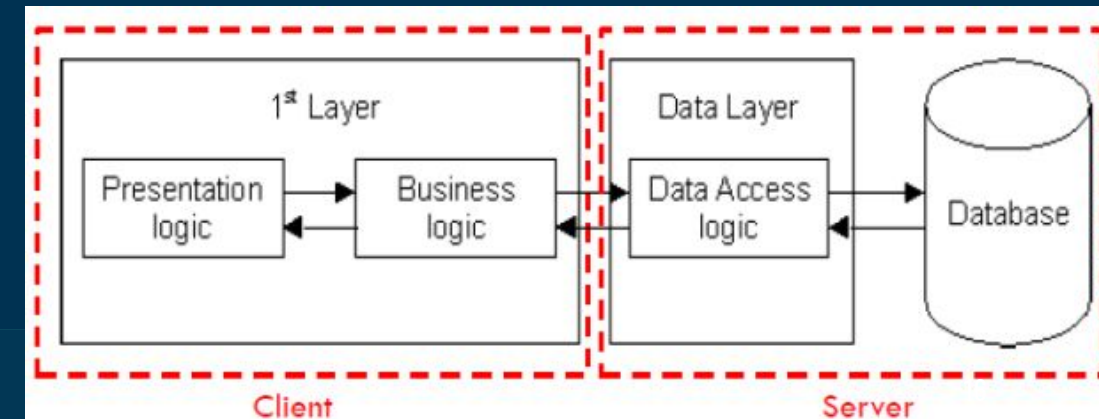
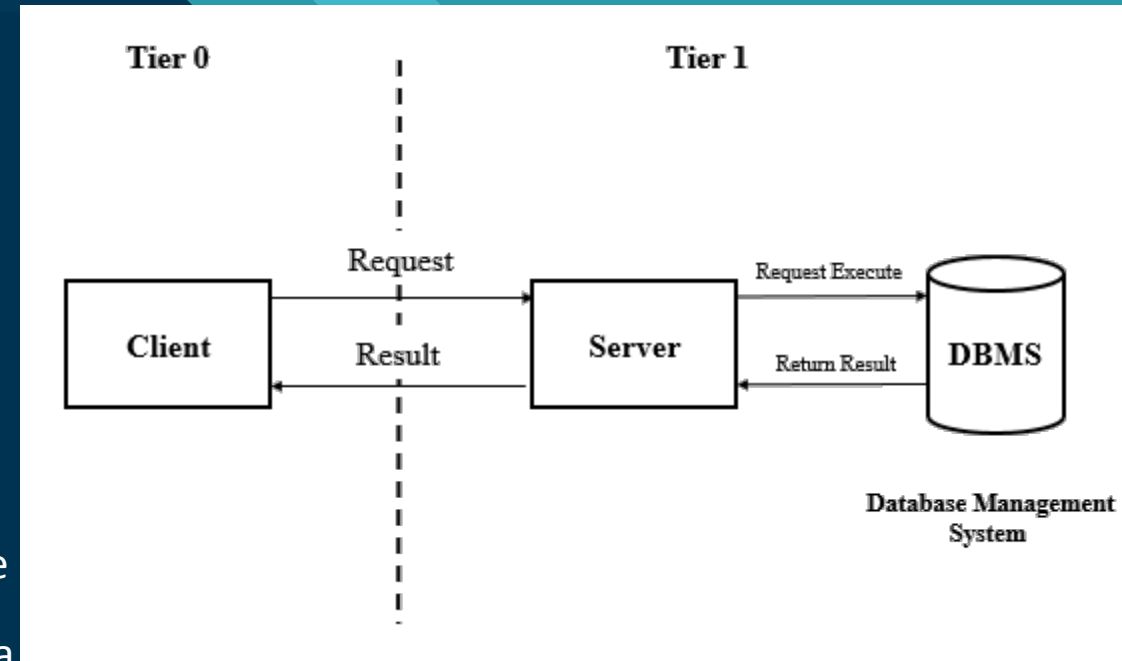
- 1 Tier Architecture in DBMS is the simplest architecture of Database in which the client, server, and Database all reside on the same machine.
- A simple one tier architecture example would be anytime you install a Database in your system and access it to practice SQL queries. But such architecture is rarely used in production.
- All 3 layers are on the same machine:
 - All code and processing kept on a single machine
 - Presentation, Logic, Data layers are tightly connected
 - Scalability: Single processor means hard to increase volume of processing
 - Portability: Moving to a new machine may mean rewriting everything
 - Maintenance: Changing one layer requires changing other layers



Client Server Architecture cont...

2. 2-Tier Architecture:

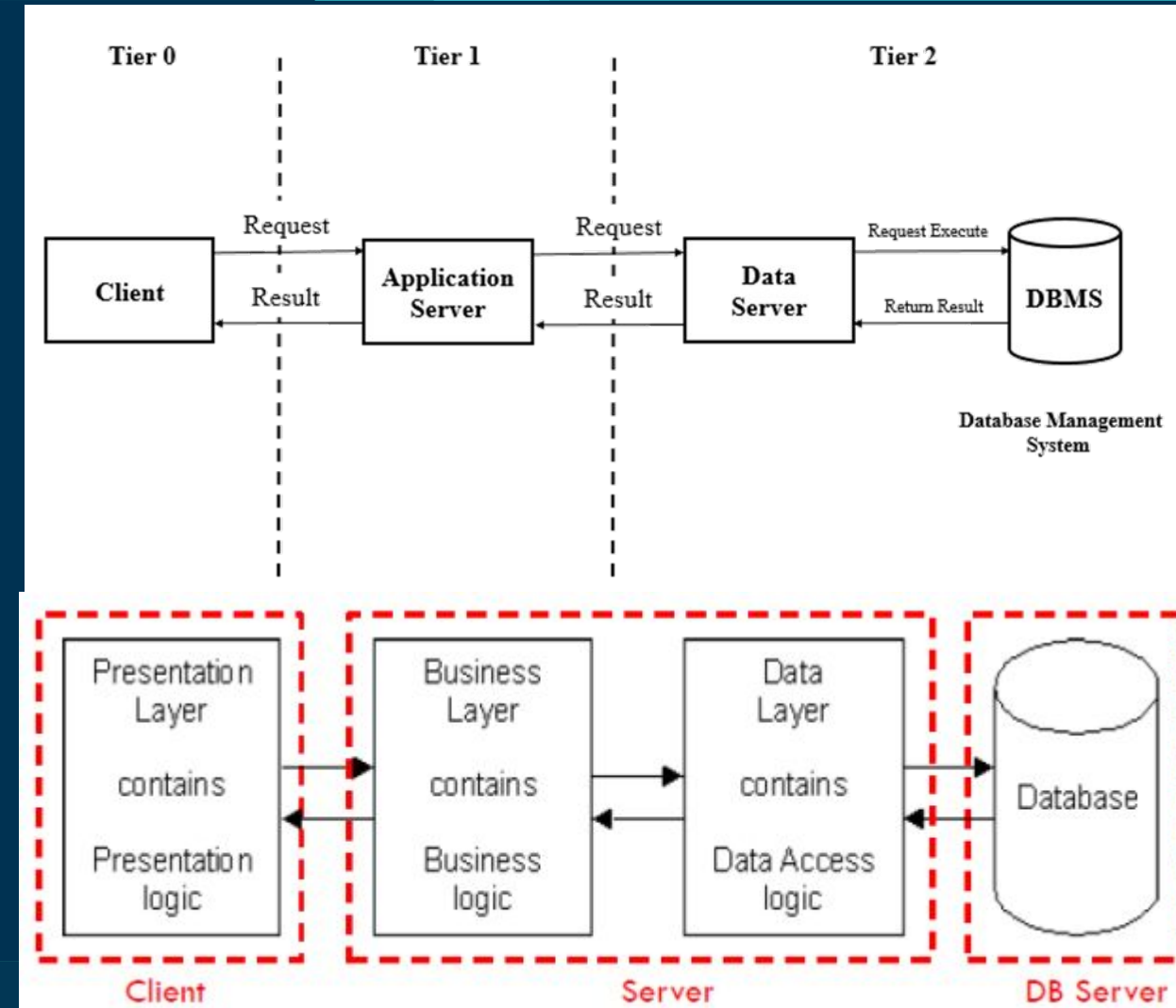
- It is used to describe client/server systems where the client requests resources and the server responds directly to the request, using its own resources.
- This means that the server does not call on another application to provide part of the service. It runs the client processes separately from the server processes, usually on a different computer:
 - The client processes provide an interface for the customer and gather and present data usually on the customer's computer. This part of the application is the presentation layer.
 - The server processes provide an interface with the data store of the business. This part of the application is the data layer.
 - The business logic that validates data, monitors security and permissions, and performs other business rules can be housed on either the client or the server or split between the two.
 - Fundamental units of work required to complete the business process.
 - Business rules can be automated by an application program.



Client Server Architecture cont...

3. 3-Tier Architecture:

- In 3-tier architecture, there is an intermediary level, meaning the architecture is generally split up between:
 1. A client, i.e., the computer, which requests the resources, equipped with a user interface (usually a web browser) for presentation purposes.
 2. The application server (also called middleware), whose task it is to provide the requested resources, but by calling on another server.
 3. The data server, which provides the application server with the data it requires



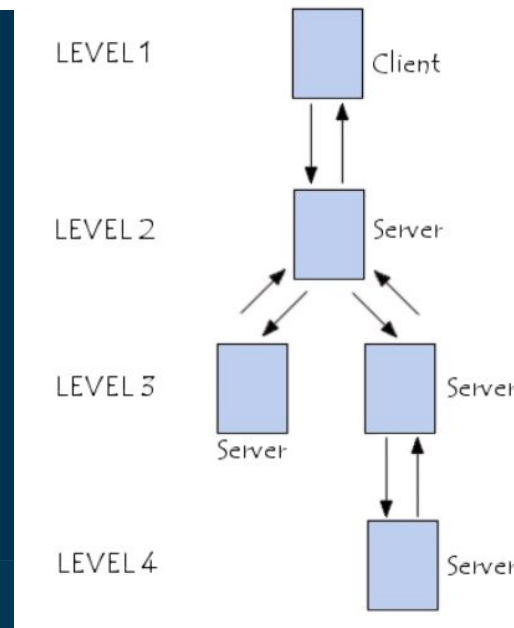
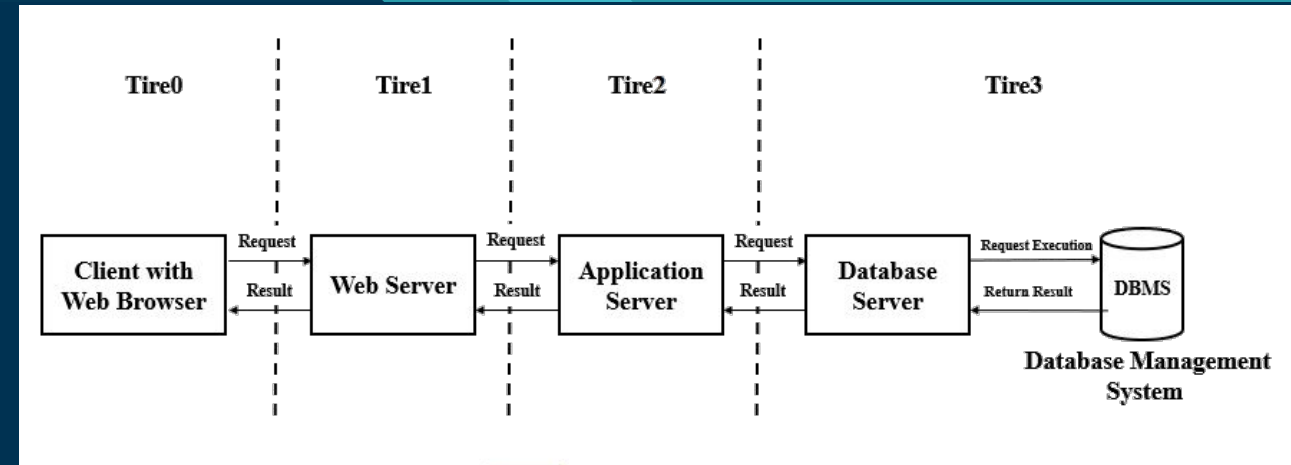
Client Server Architecture cont...

- Presentation Layer
 - Provides user interface
 - Handles the interaction with the user
 - Sometimes called the GUI or client view or front-end
 - Should not contain business logic or data access code
- Logic Layer
 - The set of rules for processing information
 - Sometimes called middleware/ back-end
 - Should not contain presentation or data access code
- Data Layer
 - The physical storage layer for data persistence
 - Manages access to DB or file system
 - Sometimes called back-end
 - Should not contain presentation or business logic code

Client Server Architecture cont...

4. N-Tier Architecture (multi-tier):

- N-tier architecture (with N more than 3) is really 3 tier architectures in which the middle tier is split up into new tiers. The application tier is broken down into separate parts
- A client, i.e., the computer, which requests the resources, equipped with a user interface (usually a web browser) for presentation purposes.
 1. The application server (also called middleware), whose task it is to provide the requested resources, but by calling on another server.
 2. The data server, which provides the application server with the data it requires



Client Server Architecture

- Advantages

1. The client/ server architecture reduces network traffic by providing a query response to the user rather than transferring total files.
2. The client/ server model improves multi-user updating through a graphical user interface (GUI) front end to the shared database.
3. Easy to implement security policies, since the data are stored in central location.
4. Simplified network administration.

- Disadvantages

1. Failure of the server causes whole network to be collapsed.
2. Expensive than P2P, dedicated powerful servers are needed
3. Extra effort is needed for administering and managing the server.

Hypertext Transfer Protocol (HTTP)

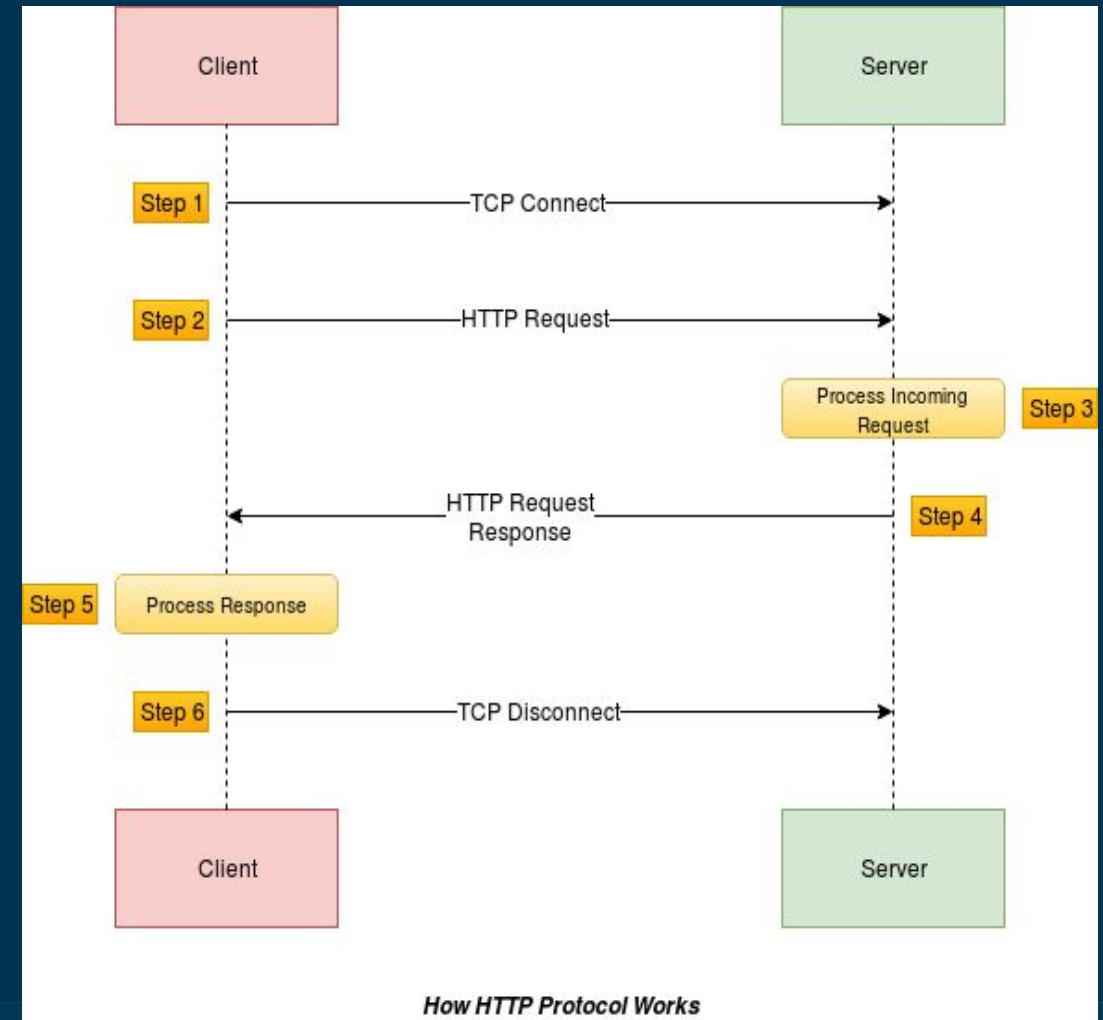
- HTTP is designed to enable communications between clients and servers.
- HTTP works as a request-response protocol between a client and server.
- Clients and servers communicate by exchanging individual messages (as opposed to a stream of data). The messages sent by the client, usually a Web browser, are called requests and the messages sent by the server as an answer are called responses.
- Normally implemented over a TCP connection (80 is standard port number for HTTP and 443 for HTTPS)

HTTP cont...

The following is the typical browser-server interaction using HTTP:

When Client enters Web address (URL) in browser and Browser uses (Domain name system) DNS to locate IP address

1. Browser opens TCP connection to server.
2. Browser sends HTTP request over connection.
3. Server processes incoming request.
4. Server sends HTTP response to browser over connection.
5. Browser processes the response and displays body of response in the client area of the browser window.
6. Browser disconnects TCP connection with the server.



HTTPS

- HTTPS (HyperText Transfer Protocol Secure) is the secure version of HTTP, adding a layer of encryption (TLS/SSL) to protect data between your browser and a website, ensuring privacy and data integrity for sensitive info like passwords and payments, indicated by a padlock icon in the address bar.
- Key Features & Benefits:
 - Encryption: Scrambles data, preventing eavesdropping and man-in-the-middle attacks.
 - Authentication: SSL/TLS certificates verify the website's identity, stopping spoofing.
 - Data Integrity: Ensures data isn't altered during transit.
 - Browser Indication: Shows a padlock icon; browsers flag HTTP as "Not Secure".
- How it Works:
 - SSL/TLS Handshake: Your browser and server exchange certificates to verify each other.
 - Key Exchange: A shared, secret key is created for encryption.
 - Secure Connection: All subsequent data is encrypted using this key, using port 443.
- Why It Matters:
 - Essential for online banking, shopping, and logins.
 - Protects personal and financial information from hackers.
 - Boosts website trust and search engine ranking (Google favors HTTPS).

Domain Names, DNS, URI

- IP addresses are not convenient for users to remember easily. So, an IP address can be represented by a natural language convention called a **Domain Name**.
- **Domain name system (DNS)** translates domain names into IP addresses. DNS is the “phone book” for the Internet, it maps between host names and IP addresses.
- Each device connected to the Internet has a unique IP address which other machines use to find the device. **DNS servers** eliminate the need for humans to memorize IP addresses.
- **Uniform Resource Identifiers (URI)** are simply formatted, case-insensitive string containing name, location, etc. to identify a resource, for example, a website, a web service, etc. A general syntax of URI used for HTTP is as follows:

URI = "http:" "://" host [":" port] [abs_path ["?" query]]

http://abc.com:80/~smith/home.html

DNS

- Domain Name System (DNS) is like a giant phonebook for the internet, converting human-friendly domain names (like `www.example.com`) into machine-friendly IP addresses (like `192.168.0.1`).
- It's like a translator ensuring that when you type in a website address, our computer knows exactly where to find it.
- The Domain Name System (DNS) turns domain names into IP addresses, which browsers use to load internet pages.
- Every device connected to the internet has its own IP address, which is used by other devices to locate the device.
- DNS servers make it possible for people to input normal words into their browsers, such as `google.com`, without having to keep track of the IP address for every website.

DNS

- What is a DNS server?
 - The Domain Name System (DNS) is the phonebook of the Internet.
 - When users type domain names such as 'google.com' or 'example.com' into web browsers, DNS is responsible for finding the correct IP address for those sites.
 - Browsers then use those addresses to communicate with origin servers or CDN edge servers to access website information.
 - This all happens thanks to DNS servers: machines dedicated to answering DNS queries.
- How DNS works?
 1. You type a domain name: You enter a website address, like `www.example.com`, into your web browser.
 2. Your computer asks a DNS server: Your computer sends a query to a DNS server, asking for the IP address associated with that domain name.
 3. The DNS server finds the IP address: The DNS server looks up the domain name in its records and finds the corresponding numerical IP address.
 4. The browser connects to the site: The DNS server returns the IP address to your computer, and your browser uses this number to connect to the web server hosting the website.

DNS

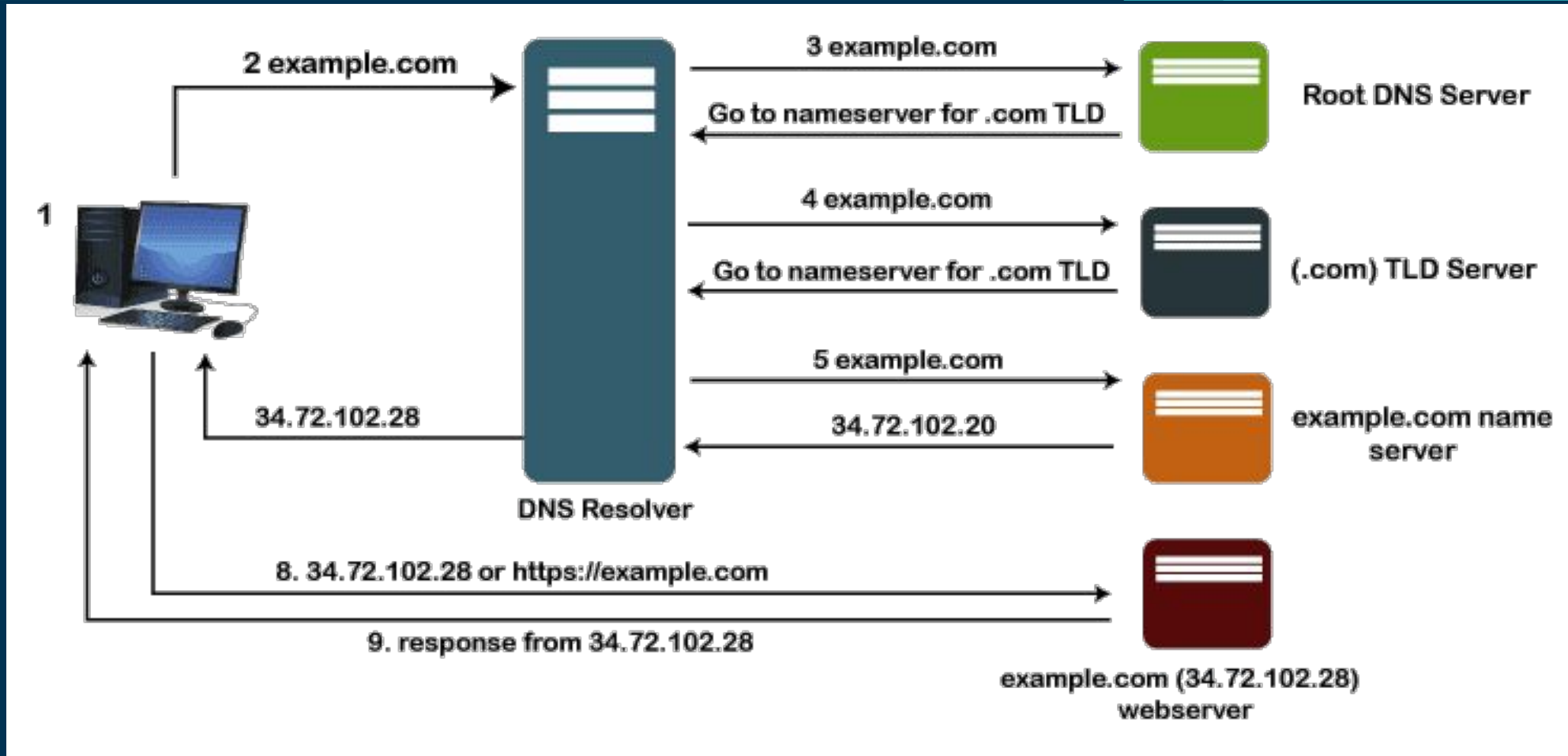


Fig: How DNS Translates Names into IP Addresses:

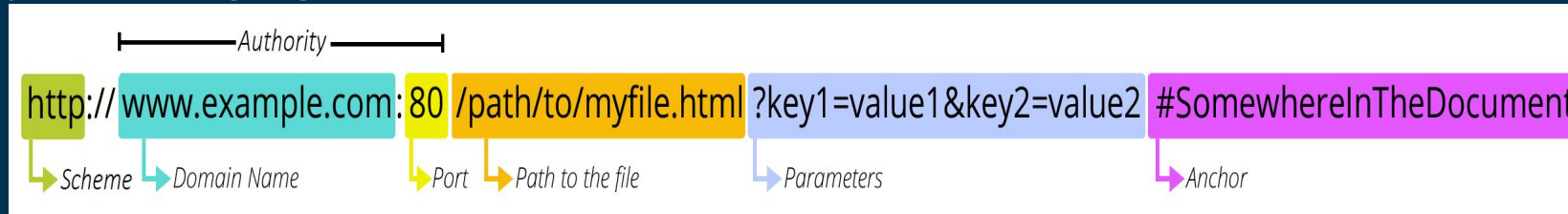
DNS and IP

Core IP Commands

Task	Windows	Linux	macOS
View IP Address	ipconfig	ip addr show or ifconfig	ifconfig or ip addr show
View all details	ipconfig /all	ifconfig -a	ifconfig -a
Test connectivity	ping	ping	ping
Trace route	tracert	tracert	tracert
View network stats	netstat	netstat or ss	netstat
DNS lookup	nslookup	nslookup or dig	nslookup or dig
View ARP table	arp -a	arp -a	arp -a

URLs

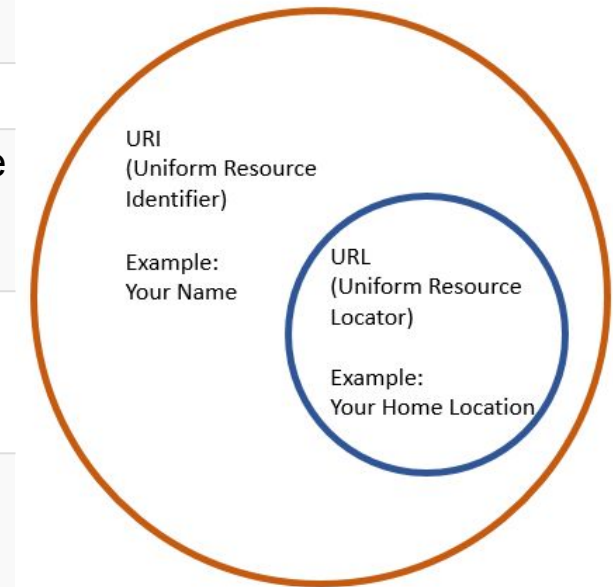
- A uniform resource locator (URL), which is the address used by a Web browser to identify the location of content on the Web, also uses a domain name as part of the URL.
 - A URL is composed of different parts, some mandatory and others optional. The most important parts are highlighted on the URL below:



- Scheme, which indicates the protocol that the browser must use to request the resources. Usually for websites the protocol is HTTPS or HTTP (its unsecured version). Browsers also know how to handle other schemes such as mailto: (to open a mail client) or ftp: to handle file transfer.
- Authority, which is separated from the scheme by the character pattern `://`. If present the authority includes both the domain (e.g. `www.example.com`) and the port (`80`), separated by a colon
- `/path/to/myfile.html` is the path to the resource on the Web server. In the early days of the Web, a path like this represented a physical file location on the Web server. Nowadays, it is mostly an abstraction handled by Web servers without any physical reality.
- `?key1=value1&key2=value2` are extra parameters provided to the Web server. Those parameters are a list of key/value pairs separated with the `&` symbol. The Web server can use those parameters to do extra stuff before returning the resource.
- `#SomewhereInTheDocument` is an anchor to another part of the resource itself. An anchor represents a sort of "bookmark" inside the resource, giving the browser the directions to show the content located at that "bookmarked" spot.

Difference between URI and URLs

URL	URI
URL stands for Uniform Resource Locator.	URI stands for Uniform Resource Identifier.
URL is a subset of URI that specifies where a resource exists and the mechanism for retrieving it.	A URI is a superset of URL that identifies a resource either by URL or URN (Uniform Resource Name) or both.
The main aim is to get the location or address of a resource	The main aim of URI is to find a resource and differentiate it from other resources using either name or location.
URL is used to locate only web pages	Used in HTML, XML and other files XSLT (Extensible Stylesheet Language Transformations) and more.
The scheme must be a protocol like HTTP, FTP, HTTPS, etc.	In URI, the scheme may be anything like a protocol, specification, name, etc.
Protocol information is given in the URL.	There is no protocol information given in URI.
Example of URL: https://google.com	Example of URI: urn:isbn:0451450523, mailto:abc@example.com, tel:+1-816-555-1212
It contains components such as protocol, domain, path, hash, query string, etc.	It contains components like scheme, authority, path, query, fragment component, etc.
All URLs can be URIs	Not all URIs are URLs since a URI can be a name instead of a locator.



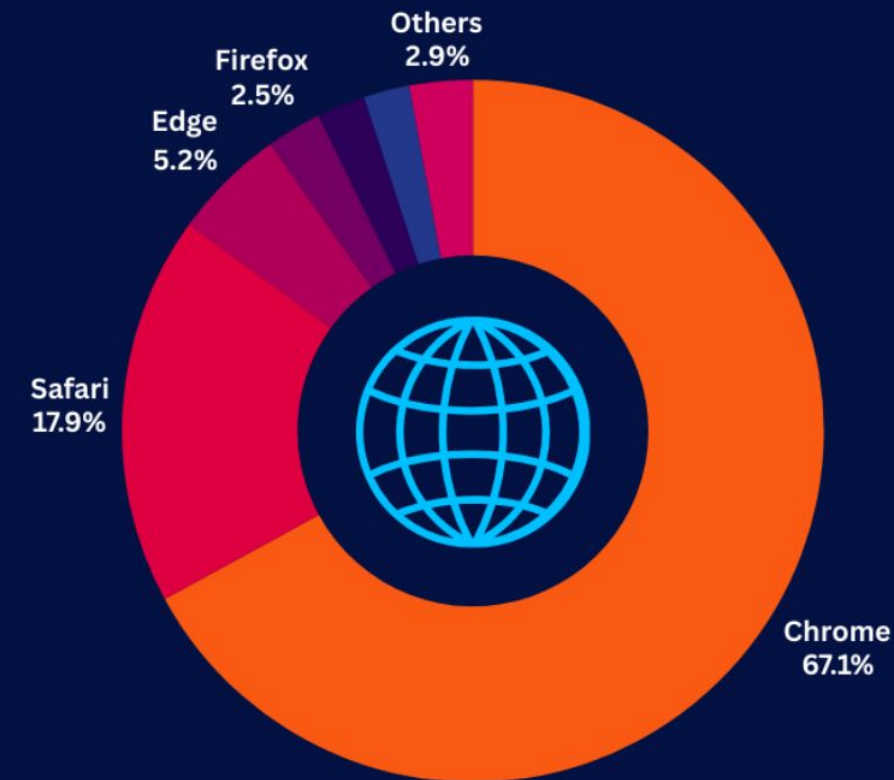
1.2 Web browser

- A browser is an application used to access and view information on the internet by displaying web pages.
- It retrieves files from web servers, translates code like HTML, and presents it as text, images, and videos.
- Popular examples include Google Chrome, Mozilla Firefox, and Safari.
- What a browser does
 - Accesses the internet: A browser acts as the interface between you and the web, allowing you to find and view content online.
 - Renders web pages: It uses a rendering engine to interpret code, such as HTML, and display the content in a readable format.
 - Displays content: It shows you websites with text, images, and videos. Multimedia, like videos, may sometimes require additional software.
 - Uses URLs: You navigate to websites by typing a unique web address, a Uniform Resource Locator (URL), into the address bar.
 - Allows navigation: You can follow links to move between different pages and websites.

1.2 Web browser (cont...)

- Key Functions of a Web Browser
 - Accessing Information: An entry of a URL in the address bar in the browser prompts the browser to make requests to the web server of the respective web page.
 - Contents Rendering: Processing the data retrieved by the browser interprets it into readable and understandable formats via HTML-CSS as text.
 - User Interface Features: Browsers also have these very simple tools, such as an address bar for navigating, buttons that go back and forth through visited pages, tabs for many open web pages, as well as bookmarks that save one's favorite sites.
 - Networking: The browser uses the client-server model that considers a browser to be a client and puts a request using HTTP or HTTPS for a data request at the server.
 - Security: HTTPS keeps secure mutual communication between the browser and the server, and it keeps sensitive information private during an interaction.

Global Browser Market Share January 2025



Commonly used ports

1. HTTP – Port 80
2. HTTPS (http with Secure Sockets Layer) – 443
3. FTP – 21
4. FTPS / SSH – 22
5. POP3 – 110
6. POP3 SSL – 995
7. IMAP – 143
8. IMAP SSL – 993
9. SMTP – 25 (Alternate: 26)
10. SMTP SSL – 587
11. MySQL – 3306
12. cPanel – 2082
13. cPanel SSL – 2083
14. WHM (Webhost Manager) – 2086
15. WHM (Webhost Manager) SSL – 2087
16. Webmail – 2095
17. Webmail SSL – 2096

1.3 Html basics: Syntax, tags, attributes

- HTML (HyperText Markup Language) is the standard markup language used to create and structure web pages.
- It defines the layout of a webpage using elements and tags, allowing for the display of text, images, links, and multimedia content.
- As the foundation of nearly all websites, HTML is used in over 95% of all web pages today, making it an essential part of modern web development.
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

1.3 Html basics: Syntax, tags, attributes

Basic HTML Tags for Document Structure

Tags	Descriptions
<html>	Encloses the entire HTML document, serving as the root element for all HTML content.
<head>	Contains header information about the webpage, including title, meta tags, and linked stylesheets. It is part of the document's structure but is not displayed on the webpage.
<title>	Used within the <head> section to define the title of the HTML document. It appears in the browser tab or window and provides a brief description of the webpage's content.
<body>	Encloses the visible content of the webpage, such as text, images, audio, videos, and links. All elements within this tag are displayed on the actual webpage when viewed in a browser.

```
<html>
  <head>
    <title>Website Title</title>
  </head>
  <body>
    <h1>Hello Geeks</h1>
    <h2>HTML Basic Structure</h2>
    <p>Paragraph Tag</p>
  </body>
</html>
```

HTML Tag Reference

Tag Description

<!--...--> Specifies a comment

<!DOCTYPE html> Specifies the document type

<a> Specifies an anchor

<abbr> Specifies an abbreviation

<acronym> Specifies an acronym

<address> Specifies an address element

<applet> Deprecated. Specifies an applet

<area> Specifies an area inside an image map

<article> Specifies an article HTML-5

<aside> Specifies some content loosely related to the page content. If it is removed, the remaining content still makes sense HTML-5

<audio> Specifies a sound content HTML-5

 Specifies bold text

<base> Specifies a base URL for all the links in a page

<basefont> Deprecated. Specifies a base font

<bdo> Specifies the direction of text display

<bdi> Represents text that must be isolated from its surrounding for bidirectional text formatting. It allows embedding a span of text with a different, or unknown, directionality HTML-5

<bgsound> Specifies background music

HTML Tag Reference cont...

<big>	Specifies big text	
<blink>	Specifies a text which blinks	
<blockquote>	Specifies a long quotation	
<body>	Specifies the body element	
 	Inserts a single line break	
<button>	Specifies a push button	
<canvas>	For making graphics with a script	HTML-5
<caption>	Specifies a table caption	
<center>	Deprecated. Specifies centered text	
<cite>	Specifies a citation	
<code>	Specifies computer code text	
<col>	Specifies attributes for table columns	
<colgroup>	Specifies groups of table columns	
<comment>	Puts a comment in the document	
<datalist>	A list of options for input values	HTML-5
<dd>	Specifies a definition description	
	Specifies deleted text	
<dfn>	Specifies a definition term	
<dialog>	Specifies a dialog box or window	

HTML Tag Reference cont...

<dir>	Deprecated. Specifies a directory list	
<div>	Specifies a section in a document	
<dl>	Specifies a definition list	
<dt>	Specifies a definition term	
	Specifies emphasized text	
<embed>	Specifies a container for an external (non-HTML) application	HTML-5
<fieldset>	Specifies a fieldset	
<figcaption>	Specifies a caption for a <figure> element	HTML-5
<figure>	Specifies self-contained content	HTML-5
	Deprecated. Specifies text font, size, and color	
<footer>	Specifies a footer for a document or section	HTML-5
<form>	Specifies a form	
<frame>	Specifies a sub window (a frame)	
<frameset>	Specifies a set of frames	
<h1> to <h6>	Specifies header 1 to header 6	
<head>	Specifies information about the document	
<header>	Specifies a header for a document or section	HTML-5
<hr>	Specifies a horizontal rule	

HTML Tag Reference cont...

<html>	Specifies an html document
<i>	Specifies italic text
<iframe>	Specifies an inline sub window (frame)
<ilayer>	Specifies an inline layer
	Specifies an image
<input>	Specifies an input field
<ins>	Specifies inserted text
<isindex>	Deprecated. Specifies a single-line input field
<kbd>	Specifies keyboard text
<keygen>	Generate key information in a form HTML-5
<label>	Specifies a label for a form control
<layer>	Specifies a layer
<legend>	Specifies a title in a fieldset
	Specifies a list item
<link>	Specifies a resource reference
<main>	Specifies the main or important content in the document. There is only one element in the document HTML-5
<map>	Specifies an image map
<mark>	Specifies a text highlighted for reference purposes, that is for its relevance in another context HTML-5
<marquee>	Creates a scrolling-text marquee

HTML Tag Reference cont...

- `<menu>` Deprecated. Specifies a menu list
- `<menuitem>` Specifies a command/menu item that the user can invoke from a popup menu HTML-5
- `<meta>` Specifies meta data of an html document which is not displayed on the page
- `<meter>` Specifies a scalar measurement within a known range (a gauge)
- `<multicol>` Specifies a multicolumn text flow
- `<nav>` Specifies a section that contains only navigation links HTML-5
- `<nobr>` No breaks allowed in the enclosed text
- `<noembed>` Specifies content to be presented by browsers that do not support the `<embed>` tag
- `<noframes>` Specifies a noframe section
- `<noscript>` Specifies a noscript section
- `<object>` Specifies an embedded object
- `<ol>` Specifies an ordered list
- `<optgroup>` Specifies an option group
- `<option>` Specifies an option in a drop-down list
- `<output>` Specifies the result of a calculation HTML-5
- `<p>` Specifies a paragraph
- `<param>` Specifies a parameter for an object
- `<plaintext>` Deprecated. Render the remainder of the document as preformatted plain text
- `<pre>` Specifies preformatted text

HTML Tag Reference cont...

<progress>	Specifies a completion progress of a task	HTML-5
<q>	Specifies a short quotation	
<rp>	Specifies to show browsers that do not support the ruby element	HTML-5
<rt>	Specifies an text ruby annotation	HTML-5
<ruby>	Specifies an ruby annotation	HTML-5
<s>	Deprecated. Specifies strikethrough text	
<samp>	Specifies sample computer code	
<script>	Specifies a script	
<section>	Specifies a section in a document	HTML-5
<select>	Specifies a selectable list	
<spacer>	Specifies a white space	
<small>	Specifies small text	
<source>	Specifies a media resources for media elements, defined inside video or audio elements	HTML-5
	Specifies a section in a document	
<strike>	Deprecated. Specifies strikethrough text	
	Specifies strong text	
<style>	Specifies a style definition	
<sub>	Specifies subscripted text	
<summary>	Specifies a summary, caption, or legend for a given <details>	HTML-5
<sup>	Specifies superscripted text	

HTML Tag Reference cont...

`<table>` Specifies a table

`<tbody>` Specifies a table body

`<td>` Specifies a table cell

`<textarea>` Specifies a text area

`<tfoot>` Specifies a table footer

`<th>` Specifies a table heading

`<thead>` Specifies a table header

`<time>` Specifies a date and time `<details>` HTML-5

`<title>` Specifies the document title

`<tr>` Specifies a table row

`<track>` Specifies a text tracks used in mediaplayers HTML-5

`<tt>` Specifies teletype text

`<u>` Deprecated. Specifies underlined text

`<ul>` Specifies an unordered list

`<var>` Specifies a variable

`<video>` Specifies a text tracks used in media players HTML-5

`<wbr>` Indicates a potential word break point within a `<nobr>` section

`<xmp>` Deprecated. Specifies preformatted text

HTML Global Attributes

Attribute	Description
accesskey	Specifies a shortcut key to activate/focus an element
class	Specifies one or more classnames for an element (refers to a class in a style sheet)
contenteditable	Specifies whether the content of an element is editable or not
data-*	Used to store custom data private to the page or application
dir	Specifies the text direction for the content in an element
draggable	Specifies whether an element is draggable or not
hidden	Specifies that an element is not yet, or is no longer, relevant
id	Specifies a unique id for an element
lang	Specifies the language of the element's content
spellcheck	Specifies whether the element is to have its spelling and grammar checked or not
style	Specifies an inline CSS style for an element
tabindex	Specifies the tabbing order of an element
title	Specifies extra information about an element
translate	Specifies whether the content of an element should be translated or not

Block-level and Inline elements

- Block-level elements

- A block-level element always starts on a new line.
- A block-level element always takes up the full width available (stretches out to the left and right as far as it can).
- A block level element has a top and a bottom margin, whereas an inline element does not.
- Here are the block-level elements in HTML:

`<address> <article><aside> <blockquote> <canvas> <dd> <div> <dl><dt>
<fieldset> <figcaption> <figure> <footer> <form> <h1>-<h6> <header> <hr>
 <main> <nav> <noscript> <p> <pre> <section> <table> <tfoot>
<video>`

- Example: `<div>Hello World</div>`

- Inline elements

- An inline element does not start on a new line.
- An inline element only takes up as much width as necessary.
- Here are the inline elements in HTML:

`<a> <abbr> <acronym> <bdo> <big>
 <button> <cite> <code>
<dfn> <i> <input> <kbd> <label> <map> <object> <output> <q>
<samp> <script> <select> <small> <sub> <sup> <textarea> <time>
<tt> <var>`

- Example: `<span>Hello World</span>`

HTML Forms

- HTML Forms are required to collect different kinds of user inputs, such as contact details like name, email address, phone numbers, or details like credit card information, etc.
- Forms contain special elements called controls like inputbox, checkboxes, radio-buttons, submit buttons, etc. Users generally complete a form by modifying its controls e.g. entering text, selecting items, etc. and submitting this form to a web server for further processing.
- The `<form>` tag is used to create an HTML form. Here's a simple example of a login form:

```
<form>
```

```
  <label>Username: <input type="text"></label>
```

```
  <label>Password: <input type="password"></label>
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

HTML Forms; Input Element

- Input Element

- This is the most used element within HTML forms.
- It allows, to specify various types of user input fields, depending on the type attribute. An input element can be of type text field, password field, checkbox, radio button, submit button, reset button, file select box, as well as several new input types introduced in HTML5.

The most frequently used input types are described below.

- Text Fields:

- Text fields are one-line areas that allow the user to input text.
- Single-line text input controls are created using an `<input>` element, whose type attribute has a value of text. Here's an example of a single-line text input used to take username:

```
<form>  
  <label for="username">Username:</label>  
  <input type="text" name="username" id="username">  
</form>
```



Username:

HTML Forms; password

- Password

- Password fields are like text fields. The only difference is; characters in a password field are masked, i.e. they are shown as asterisks or dots. This is to prevent someone else from reading the password on the screen.
- This is also a single-line text input controls created using an `<input>` element whose type attribute has a value of password.

Here's an example of a single-line password input used to take user password:

```
<form>
```

```
  <label for="user-pwd">Password:</label>
```

```
  <input type="password" name="user-password" id="user-pwd">
```



HTML Forms; Radio Buttons

- Radio Buttons

- Radio buttons are used to let the user select exactly one option from a pre-defined set of options. It is created using an `<input>` element whose type attribute has a value of radio.
- Here's an example of radio buttons that can be used to collect user's gender information:

```
<form>  
  <input type="radio" name="gender" id="male">  
  <label for="male">Male</label>  
  <input type="radio" name="gender" id="female">  
  <label for="female">Female</label>  
</form>
```



☐ Male ☐ Female

HTML Forms; Checkboxes

- Checkboxes

- Checkboxes allows the user to select one or more option from a pre-defined set of options. It is created using an `<input>` element whose type attribute has a value of checkbox.
- Here's an example of checkboxes that can be used to collect information about user's hobbies:

```
<form>
```

```
  <input type="checkbox" name="sports" id="soccer">
```

```
  <label for="soccer">Soccer</label>
```

```
  <input type="checkbox" name="sports" id="cricket">
```

```
  <label for="cricket">Cricket</label>
```

```
  <input type="checkbox" name="sports" id="baseball">
```

```
  <label for="baseball">Baseball</label>
```

```
</form>
```



☐ Soccer ☐ Cricket ☐ Baseball

HTML Forms; File Select box

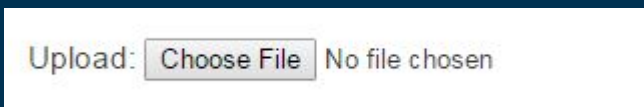
- File Select box
 - The file fields allow a user to browse for a local file and send it as an attachment with the form data. Web browsers such as Google Chrome and Firefox render a file select input field with a Browse button that enables the user to navigate the local hard drive and select a file.
 - This is also created using an `<input>` element, whose type attribute value is set to file.

```
<form>
```

```
  <label for="file-select">Upload:</label>
```

```
  <input type="file" name="upload" id="file-select">
```

```
</form>
```



HTML Forms; Textarea

- Textarea

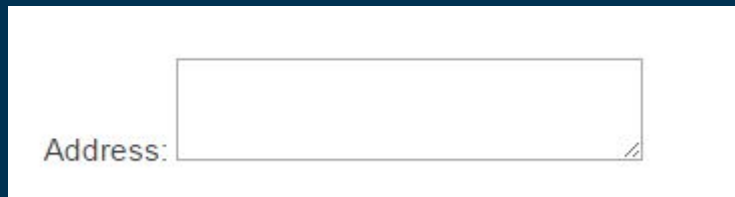
- Textarea is a multiple-line text input control that allows a user to enter more than one line of text.
- Multi-line text input controls are created using an `<textarea>` element.

```
<form>
```

```
  <label for="address">Address:</label>
```

```
  <textarea rows="3" cols="30" name="address" id="address"></textarea>
```

```
</form>
```



Address:

HTML Forms; Select Boxes

- Select Boxes

- A select box is a dropdown list of options that allows user to select one or more option from a pull-down list of options. Select box is created using the `<select>` element and `<option>` element.
- The `<option>` elements within the `<select>` element define each list item.

```
<form>
```

```
  <label for="city">City:</label>
```

```
  <select name="city" id="city">
```

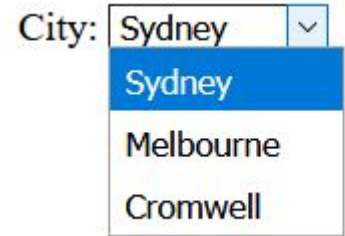
```
    <option value="sydney">Sydney</option>
```

```
    <option value="melbourne">Melbourne</option>
```

```
    <option value="cromwell">Cromwell</option>
```

```
  </select>
```

```
</form>
```



City: Sydney ▼

- Sydney
- Melbourne
- Cromwell

HTML Forms; Submit and Reset Buttons

- Submit and Reset Buttons

- A submit button is used to send the form data to a web server. When submit button is clicked the form data is sent to the file specified in the form's action attribute to process the submitted data.
- A reset button resets all the forms control to default values. Try out the following example by typing your name in the text field and click on submit button to see it in action.

```
<form action="action.php" method="post">  
  <label for="first-name">First Name:</label>  
  <input type="text" name="first-name" id="first-name">  
  <input type="submit" value="Submit">  
  <input type="reset" value="Reset">  
</form>
```



First Name:

Grouping Form Controls

- Grouping Form Controls

- We also group logically related controls and labels within a web form using the `<legend>` element. Grouping form controls into categories makes it easier for users to locate a control which makes the form more user-friendly.
- Let's try out the following example to see how it works:

```
<form>
```

```
  <fieldset>
```

```
    <legend>Contact Details</legend>
```

```
    <label>Email Address: <input type="email" name="email"></label>
```

```
    <label>Phone Number: <input type="text" name="phone"></label>
```

```
  </fieldset>
```



Contact Details

Email Address: Phone Number:

Labeling input files

- Associating a `<label>` with an `<input>` element offers some major advantages:
 - The label text is not only visually associated with its corresponding text input; it is programmatically associated with it too. This means that, for example, a screen reader will read out the label when the user is focused on the form input, making it easier for an assistive technology user to understand what data should be entered.
 - We can click the associated label to focus/activate the input, as well as the input itself. This increased hit area provides an advantage to anyone trying to activate the input, including those using a touch-screen device.
- To associate the `<label>` with an `<input>` element, we need to give the `<input>` an **id** attribute. The `<label>` then needs a **for** attribute whose value is the same as the **input's id**.

```
<label for="abc">Do you like peas?</label>
<input type="checkbox" name="pea" id="abc">
```
- Alternatively, you can nest the `<input>` directly inside the `<label>`, in which case the `for` and `id` attributes are not needed because the association is implicit:

```
<label>Do you like peas?
  <input type="checkbox" name="peas">
</label>
```

- Attribute

for

- The `id` of a labelable form-related element in the same document as the `<label>` element. The first element in the document with an `id` matching the value of the `for` attribute is the labeled control for this label element if it is a labelable element. If it is not labelable then the `for` attribute has no effect. If there are other elements that also match the `id` value, later in the document, they are not considered.

Frequently Used HTML Form Attributes

Attribute	Description
name	Specifies the name of the form.
action	Specifies the URL of the program or script on the web server that will be used for processing the information submitted via form.
method	Specifies the HTTP method used for sending the data to the web server by the browser. The value can be either get (the default) and post.
target	Specifies where to display the response that is received after submitting the form. Possible values are _blank, _self, _parent and _top.
enctype	Specifies how the form data should be encoded when submitting the form to the server. Applicable only when the value of the method attribute is post.

HTML Forms and Scripting

- Most webmasters need some customizations to their forms, and this is where JavaScript comes in handy.
- JavaScript performs client-side dynamic calls to forms and the input typed by users.
- For instance, suppose we want to ensure that the user has entered a value into a "first_name" text box before we send the input to the server. We could send the input to the server, process the data, find the missing input variable, and then send the form back to the user with an error message. This process takes time, so we can do a quick check for an empty first name text box using JavaScript. This is just one of the many reasons for using JavaScript validation in your HTML forms.

HTML Form basics

- To create a form in HTML, we use the `<form>` element:

```
<form action="/signup" method="post" id="signup">  
</form>
```

- The `<form>` element has two important attributes: `action` and `method`
 1. **action** is a URL that will process the form submission. In this case, it is `/signup`.
 2. **method** is the HTTP method to submit the form with. Typically, it is the `post` or `get` method. The `post` method sends data to the server as the request body while the `get` method appends the form data to the action URL with a `?` operator.
- In JavaScript, the form is represented as the `HTMLFormElement` object. The `HTMLFormElement` has the following corresponding properties:
 - `action` – the URL that will process the form data. It is equivalent to the HTML `action` attribute
 - `method` – can be `get` or `post`, which is equivalent to the HTML `method` attribute.
- The `HTMLFormElement` element also provides the following useful methods:
 - `submit()` – submit the form.
 - `reset()` – reset the form.

HTML Referencing forms

- To reference the <form> element, we can use the getElementById() method if the form has an id attribute:
let form = document.getElementById('subscribe');
Let form = document.getElementById(test');
- An HTML document can have multiple forms, not just one. The document.forms property returns a collection of forms on the document:
document.forms
- We can reference each form by numeric index. The following statement returns the first form in the document:
document.forms[1]

Submitting HTML forms

- Typically, a form has a submit button so that when we click it, the form data is sent to the action URL on the server for further processing.

- To create a submit button, we use `<input>` or `<button>` element with the type submit:

```
<input type="submit" value="Subscribe">
```

Or

```
<button type="submit">Subscribe</button>
```

- If the submit button has focus and we press the Enter keyboard, the form is also submitted.
- When we submit the form this way, the submit event fires right before the request is sent to the server. It gives you the chance to validate the form data and decide whether to continue to submit the form or not.
- To attach an event listener to the submit event, we use the `addEventListener()` method:

```
let form = document.getElementById('signup');  
form.addEventListener('submit', (event) => {  
    // handle the form data  
});
```

- If we want to stop the form from being submitted, we call the `preventDefault()` method inside the submit event listener:

```
form.addEventListener('submit', (event) => { // stop form submission  
    event.preventDefault();  
});
```

- Generally, we call the `event.preventDefault()` method if the form is invalid.
- To submit the form from JavaScript, we call the `submit()` method:

```
form.submit()
```

Note that the `form.submit()` does not cause the submit event to fire, therefore, we should validate data before calling this method.

HTML Forms and Scripting cont...

Getting a value from a form object

- Example:
 - Load the page, then type something into the text box. Click the button, and what we typed is shown in the alert box.
 - Here's how the script works.
- 1. JavaScript calls the testResults function when we click the button in the form.
- 2. The testResults function is passed the form object using the syntax this.form (this keyword references the button control; form is a property of the button control and represents the form object). We have given the form object the name "form" inside the testResult function, but can give any name.
- 3. The testResults function is simple -- it merely copies the contents of the text box to a variable named TestVar. Notice how the text box contents was referenced. we defined the form object we wanted to use (called form), the object within the form that we wanted (called inputbox), and the property of that object we wanted (the value property).

```
<html>
  <head>
    <title>test input</title>
    <script language="javascript">
      function testresults (form) {
        var testvar = form.inputbox.value;
        alert ("you typed: " + testvar);
      }
    </script>
  </head>
  <body>
    <form name="myform" action=""
      method="get">enter something in the box:
    <br>
      <input type="text" name="inputbox" value="">
      <input type="button" name="button"
        value="click" onclick="testresults(this.form)">
    </form>
  </body>
</html>
```

HTML Forms and Scripting cont...

Setting a value in a form object

- Example:
 - The process of setting the value in a form object is just the reverse of reading it. Here's a short example to demonstrate setting a value in a form text box. The process is like the previous example, except this time there are two buttons. Click the "Read" button and the script reads what you typed into the text box. Click the "Write" button and the script writes a particular phrase into the text box.

 1. When we click the "Read" button, JavaScript calls the readText function, which reads and displays the value we entered into the text box.
 2. When we click the "Write" button, JavaScript calls the writeText function, which writes "Have a nice day!" in the text box.

```
<html>
  <head>
    <title>test input </title>
    <script language="javascript">
      function readtext (form) {
        testvar =form.inputbox.value;
        alert ("you typed: " + testvar);
      }
      function writetext (form) {
        form.inputbox.value = "have a nice day!"
      }
    </script>
  </head>
  <body>
    <form name="myform" action="" method="get">
      enter something in the box: <br>
        <input type="text" name="inputbox" value="">
        <input type="button" name="button1" value="read"
          onclick="readtext(this.form)">
        <input type="button" name="button2" value="write"
          onclick="writetext(this.form)">
      </form>
    </body>
  </html>
```

Disabled and read-only fields

- HTML `<input>` disabled Attribute:
 - The disabled attribute is a boolean attribute.
 - When present, it specifies that the `<input>` element should be disabled.
 - A disabled input element is unusable and un-clickable.
 - The disabled attribute can be set to keep a user from using the `<input>` element until some other condition has been met (like selecting a checkbox, etc.). Then, a JavaScript could remove the disabled value, and make the `<input>` element usable.
 - Disabled `<input>` elements in a form will not be submitted!
 - Example:

```
<label for="lname">Last name:</label>
```

```
<input type="text" id="lname" name="lname" disabled><br><br>
```

Last name:

Disabled and read-only fields

- HTML `<input>` readonly Attribute:
 - The readonly attribute is a boolean attribute.
 - When present, it specifies that an input field is read-only.
 - A read-only input field cannot be modified (however, a user can tab to it, highlight it, and copy the text from it).
 - The readonly attribute can be set to keep a user from changing the value until some other conditions have been met (like selecting a checkbox, etc.). Then, a JavaScript can remove the readonly value, and make the input field editable.
 - A form will still submit an input field that is readonly, but will not submit an input field that is disabled!
 - Example:

```
<label for="country">Country:</label>
```

```
<input type="text" id="country" name="country" value="Norway" readonly>
```

Country:

Disabled vs read-only fields

- The ReadOnly and Disabled attributes though work in same manner i.e. restrict the User from inputting data in the Form elements in HTML, there are still differences in both.

ReadOnly Attribute

- The ReadOnly attribute as the name suggests makes the element ReadOnly i.e. the element will be enabled but its contents cannot be edited.
 1. Only some elements such as HTML INPUT elements like TextBox and TEXTAREA can be set as ReadOnly.
 2. Cursor is shown in ReadOnly TextBox or TextArea.
 3. Focus can be set in ReadOnly element.
 4. ReadOnly elements are focused during TAB navigation.
 5. Data of ReadOnly elements is sent to server during Form Submission (Form POST).
 6. Appears as normal element in Browser.

Disabled Attribute

- The Disabled attribute as the name suggests makes the element Disabled i.e. the element is disabled completely.
 1. All elements can be set as Disabled.
 2. Cursor is not shown in Disabled TextBox or TextArea.
 3. Focus cannot be set in Disabled element.
 4. Disabled elements are not focused during TAB navigation.
 5. Data of Disabled elements is not sent to server during Form Submission (Form POST).
 6. Appears greyed out in Browser.

Form field Event handlers

- A user action on any element of form is considered an event.
- This includes mouse movement, mouse clicks, typing on the keyboard and so on.
- Frequently we would want some JavaScript code to be executed in response to a user event.
- For instance, consider writing a JavaScript application that converts dollar amounts to French Francs. In such an application we might want the currency conversion function (assuming we have written one) to be executed when the user clicks on a "Convert" button on the form. That is, we want to "handle" (take care of) the response for a click of the mouse on this button.
- An "event handler" is a piece of JavaScript text that we associate with some kind category of event on a particular user interface element.
- Various user interface elements respond to various events, and not all events are "interesting" for all elements---while a mouse click on a button might signal the user's interest in the initiation of some calculation, a mouse click on a textarea element is not quite as "interesting".
- The most common way of creating event handlers is by specifying some function to be executed when an event occurs. We shall use this method in all our examples.
- For example, suppose we have created a JavaScript function called compute. You can have this function called when the user clicks a button by assigning the function call to the button's onClick event handler:

```
<INPUT TYPE="button" VALUE="Calculate" onClick="compute()">
```

Some Events

Event	Applies to	Occurs when	Event handler
Blur	windows and all form elements	User removes input focus from window or form element	onBlur
Change	text fields, textareas, select lists	User changes value of element	onChange
Click	buttons, radio buttons, checkboxes, submit buttons, reset buttons	User clicks form element or link	onClick
Focus	windows and all form elements	User gives input focus to window or form element	onFocus
KeyDown	text areas	User depresses a key	onKeyDown
KeyPress	text areas	User presses or holds down a key	onKeyPress
KeyUp	text areas	User releases a key	onKeyUp
MouseDown	documents, buttons	User depresses a mouse button	onMouseDown
MouseMove	nothing by default	User moves the cursor	onMouseMove
MouseUp	documents, buttons, links	User releases a mouse button	onMouseUp

Example onblur event handler

```
<!DOCTYPE html>
```

```
<html>
```

```
  <body>
```

```
    Enter your name: <input type="text" id="fname" onblur="myFunction()">
```

```
    <p>When you leave the input field, a function is triggered which transforms the  
    input text to upper case.</p>
```

```
    <script>
```

```
    function myFunction() {
```

```
      var x = document.getElementById("fname");
```

```
      x.value = x.value.toUpperCase();
```

```
    }
```

```
    </script>
```

```
  </body>
```

```
</html>
```

Example onchange event handler

```
<!DOCTYPE html>
<html>
  <body>
    <p>Select a new car from the list.</p>
    <select id="mySelect" onchange="myFunction()">
      <option value="Audi">Audi</option>
      <option value="BMW">BMW</option>
      <option value="Mercedes">Mercedes</option>
      <option value="Volvo">Volvo</option>
    </select>
    <p>When you select a new car, a function is triggered which outputs the value of the selected car.</p>
    <p id="demo"></p>
    <script>
      function myFunction() {
        var x = document.getElementById("mySelect").value;
        document.getElementById("demo").innerHTML = "You selected: " + x;
      }
    </script>
  </body>
</html>
```

Example onclick event handler

```
<!DOCTYPE html>
```

```
<html>
```

```
  <body>
```

```
    <h1>The onclick Event</h1>
```

```
    <p>The onclick event is used to trigger a function when an element is clicked on.</p>
```

```
    <p>Click the button to trigger a function that will output "Hello World" in a p element with  
    id="demo".</p>
```

```
    <button onclick="myFunction()">Click me</button>
```

```
    <p id="demo"></p>
```

```
    <script>
```

```
    function myFunction() {
```

```
      document.getElementById("demo").innerHTML = "Hello World";
```

```
    }
```

```
  </script>
```

```
</body>
```

```
</html>
```

Example onfocus event handler

```
<!DOCTYPE html>
```

```
<html>
```

```
  <body>
```

```
    Enter your name: <input type="text" onfocus="myFunction(this)">
```

```
    <p>When the input field gets focus, a function is triggered which changes the  
    background-color.</p>
```

```
    <script>
```

```
    function myFunction(x) {  
      x.style.background = "yellow";  
    }
```

```
    </script>
```

```
  </body>
```

```
</html>
```

Example onkeydown event handler

```
<!DOCTYPE html>
```

```
<html>
```

```
  <body>
```

```
    <p>A function is triggered when the user is pressing a key in the input field.</p>
```

```
    <input type="text" onkeydown="myFunction()">
```

```
    <script>
```

```
    function myFunction() {
```

```
      alert("You pressed a key inside the input field");
```

```
    }
```

```
    </script>
```

```
  </body>
```

```
</html>
```

Passing HTML form data

- The `<form>` element defines how the data will be sent. All its attributes are designed to let us configure the request to be sent when a user hits a submit button. The two most important attributes are **action** and **method**.
- action attribute:
 - The action attribute defines where the data gets sent. Its value must be a valid relative or absolute URL. If this attribute isn't provided, the data will be sent to the URL of the page containing the form — the current page.
 - `<form action="https://example.com">` (the data is sent to an absolute URL)
 - `<form action="/somewhere_else">` (a relative URL — the data is sent to a different URL on the same origin)
 - `<form>` (When specified with no attributes, the `<form>` data is sent to the same page that the form is present on)
 - The action value should be a file on the server that can handle the incoming data, including ensuring server-side validation. The server then responds, generally handling the data and loading the URL defined by the action attribute, causing a new page load (or a refresh of the existing page, if the action points to the same page).

Passing HTML form data cont..

- method attribute:
 - The method attribute defines how data is sent. The HTTP protocol provides several ways to perform a request; HTML form data can be transmitted via a number of different methods, the most common being the GET method and the POST method
- **The GET method:**
 - The GET method is the method used by the browser to ask the server to send back a given resource: "Hey server, I want to get this resource." In this case, the browser sends an empty body. Because the body is empty, if a form is sent using this method the data sent to the server is appended to the URL.
 - Since the GET method has been used, you'll see the URL `www.foo.com/?say=Hi&to=Mom` appear in the browser address bar when we submit the form.
 - The data is appended to the URL as a series of name/value pairs. After the URL web address has ended, we include a question mark (?) followed by the name/value pairs, each one separated by an ampersand (&). In this case we are passing two pieces of data to the server:
 - say, which has a value of Hi
 - to, which has a value of Mom
 - The HTTP request looks like this `GET /?say=Hi&to=Mom HTTP/2.0, Host: foo.com`
- Consider the following form:

```
<form action="http://www.foo.com" method="GET">
  <div>
    <label for="say">What greeting do you want to say?</label>
    <input name="say" id="say" value="Hi">
  </div>
  <div>
    <label for="to">Who do you want to say it to?</label>
    <input name="to" id="to" value="Mom">
  </div>
  <button>Send my greetings</button>
</div>
</form>
```

Passing HTML form data cont..

- **The Post method:**

- The POST method is a little different. It's the method the browser uses to talk to the server when asking for a response that considers the data provided in the body of the HTTP request: "Hey server, take a look at this data and send me back an appropriate result." If a form is sent using this method, the data is appended to the body of the HTTP request.
- When the form is submitted using the POST method, we get no data appended to the URL, and the HTTP request looks like so, with the data included in the request body instead:

POST / HTTP/2.0

Host: foo.com

Content-Type: application/x-www-form-urlencoded

Content-Length: 13

say=Hi&to=Mom

- The Content-Length header indicates the size of the body, and the Content-Type header indicates the type of resource sent to the server

- Consider the following form:

```
<form action="http://www.foo.com"
method="POST">
```

```
<div>
```

```
<label for="say">What greeting do
you want to say?</label>
```

```
<input name="say" id="say"
value="Hi">
```

```
</div>
```

```
<div>
```

```
<label for="to">Who do you want to
say it to?</label>
```

```
<input name="to" id="to"
value="Mom">
```

```
</div>
```

```
<div>
```

```
<button>Send my greetings</button>
```

```
</div>
```

```
</form>
```

1.3 Html basics: tables

Introduction To Html Tables and Their Structure

- A table is a structured set of data made up of rows and columns (tabular data).
- The HTML tables allow web authors to arrange data like text, images, links, other tables, etc. into rows and columns of cells.
- The HTML tables are created using the `<table>` tag in which the `<tr>` tag is used to create table rows and `<td>` tag is used to create data cells. The elements under `<td>` are regular and left aligned by default.

Table structure

- An HTML table has the following structure:
 - An optional caption.
 - One or more groups of rows. Each row group consists of an optional head section, an optional foot section, and a series of rows.
 - One or more groups of columns.
 - Each row consists of one or more cells.
 - Each cell may contain either header information (meant to describe the nature of data in the column or row) or data. A cell may span more than one row or column.

Html Tables

- Example

```
<table width="100%" >  
  <tr>  
    <th>Firstname</th>  
    <th>Lastname</th>  
    <th>Age</th>  
  </tr>  
  <tr>  
    <td>Jill</td>  
    <td>Smith</td>  
    <td>50</td>  
  </tr>  
  <tr>  
    <td>Eve</td>  
    <td>Jackson</td>  
    <td>94</td>  
  </tr>  
</table>
```

Alignment

- For the Alignment, align attribute is used.
- HTML align attribute supports col, colgroup, tbody, td, tfoot, th, thead, tr elements in a table.
- Type of value of HTML align attribute is predefined.

Name	Description
left	Left align data, left justify text.
center	Center align data, center justify text.
right	Right align data, right justify text.
justify	Double justify text.
char	If used, text is aligned around a specific character.

Aligning Entire Table

- The HTML `<table>` align Attribute is used to specify the alignment of the table and its content.
- It is possible to change the horizontal alignment of items within table cells. Table data defaults to left alignment; table headers to center. In order to change the alignment in one cell, insert the appropriate "ALIGN=" attribute within the code for that cell.

Syntax:

```
<table align="left | right | center">
```

Attribute Values:

- left: It sets the left align to the table.
- right: It sets the right align to the table.
- center: It sets the center align to the table.

```
<!DOCTYPE html>
<html>
<head>
  <title> HTML table align Attribute</title>
</head>
<body>
<h2>HTML table align Attribute</h2>
<table border="1" align="right">
  <caption>Author Details</caption>
  <tr>
<th>NAME</th><th>AGE</th><th>BRANCH</th>
  </tr>
  <tr>
    <td>BITTU</td><td>22</td> <td>CSE</td>
  </tr>
  <tr>
    <td>RAM</td><td>21</td><td>ECE</td>
  </tr>
</table>
</body>
</html>
```

Alignment within a Row

- It is possible to change the vertical alignment of items within table row (tr). Data will default to the vertical middle of a row unless additional code is added. To place an item at the top or bottom of row, insert the "VALIGN=" attribute within the code for that row.
- VALIGN

The VALIGN attribute can be used to explicitly specify the vertical alignment of material within a table row. It is overridden by the VALIGN attribute on row:

 - valign=top

The row item contents appear at the top of each row (the default).
 - valign=middle

Row item contents are centered vertically in each row.
 - valign=bottom

The row item contents appear at the bottom of each row.
 - valign=baseline

This is used when you want to ensure that all cells in the row share the same baseline. This constraint only applies to the first text line for each cell.
- Horizontal align can also be added in table row.
 - align= "left/center/right"

```
<!DOCTYPE html>
<html>
<head>
  <title> HTML table align Attribute</title>
</head>
<body>
<h2>HTML table align Attribute</h2>
<table border="1" align="right">
  <caption>Author Details</caption>
  <tr valign="top" align="right">
    <th><br>NAME</th><th>AGE</th><th>BRANCH</th>
  </tr>
  <tr>
    <td>BITTU</td><td>22</td> <td>CSE</td>
  </tr>
  <tr>
    <td>RAM</td><td>21</td><td>ECE</td>
  </tr>
</table>
</body>
</html>
```

Alignment within a Cell

- It is also possible to change the vertical alignment of items within table cells (td). Data will default to the vertical middle of a cell unless additional code is added. To place an item at the top or bottom of its cell, insert the "VALIGN=" attribute within the code for that cell.

VALIGN

- The VALIGN attribute can be used to explicitly specify the vertical alignment of material within a table row. It is overridden by the VALIGN attribute on individual cells:
 - `valign=top`
The cell contents appear at the top of each cell (the default).
 - `valign=middle`
Cell contents are centered vertically in each cell.
 - `valign=bottom`
The cell contents appear at the bottom of each cell.
 - `valign=baseline`
This is used when you want to ensure that all cells in the row share the same baseline. This constraint only applies to the first text line for each cell.
- Horizontal align can also be added in table cell(td).
`align= "left/center/right"`

```
<!DOCTYPE html>
<html>
<head>
  <title> HTML table align Attribute</title>
</head>
<body>
<h2>HTML table align Attribute</h2>
<table border="1" align="right">
  <caption>Author Details</caption>
  <tr>
<th><br>NAME</th><th>AGE</th><th>BRANCH</t
h>    </tr>
    <tr>
      <td><br>BITTU</td><td valign="top"
align="left">22</td> <td>CSE</td>
    </tr>
    <tr>
      <td>RAM</td><td>21</td><td>ECE</td>
    </tr>
  </table>
</body>
</html>
```

Table Attributes

Attribute	Value	Description
abbr	abbreviated_text	<i>Deprecated</i> – Specifies an abbreviated version of the content in a cell.
align	right, left, center, justify, char	<i>Deprecated</i> – Visual alignment.
bgcolor	rgb(x,x,x), #hexcode, colorname	<i>Deprecated</i> – Specifies the background color of the table.
border	pixels	<i>Deprecated</i> – Specifies the border width. A value of "0" means no border.
cellpadding	pixels or %	<i>Deprecated</i> – Specifies the space between the cell borders and their contents.
cellspacing	pixels or %	<i>Deprecated</i> – Specifies the space between cells.
frame	void, above, below, hside, lhs, rhs, vside, box, border	<i>Deprecated</i> – Used in conjunction with the border attribute, specifies which side of the frame that makes up the border surrounding the table is displayed.
rules	none, groups, rows, cols, all	<i>Deprecated</i> – Used in conjunction with the border attribute, specifies which rules appear between the cells of the table.
summary	text	<i>Deprecated</i> – Specifies the summary of the content.
width	pixels or %	<i>Deprecated</i> – Specifies the width of the table.
height	pixels or %	<i>Deprecated</i> – Specifies the height of the table.

Content Summary

- The summary attribute specifies a summary of the content of a table.
- The summary attribute makes no visual difference in ordinary web browsers.
- This attribute can be used by screen readers.

- Example

```
<table border="1" summary="Monthly savings for the Flintstones family">  
  <tr>  
    <th>Month</th>  
    <th>Savings</th>  
  </tr>  
  <tr>  
    <td>January</td>  
    <td>$100</td>  
  </tr>  
</table>
```

Background Color

- The bgcolor attribute specifies a background color of a table. It can also be added in table row and table cell

- Example

```
<table border="1" summary="Monthly savings for the Flintstones family"
bgcolor="red">
  <tr bgcolor="white">
    <th>Month</th>
    <th>Savings</th>
  </tr>
  <tr>
    <td bgcolor="green">January</td>
    <td>$100</td>
  </tr>
</table>
```

- CSS background-color property has replaced bgcolor attribute.
 - bgcolor="red" can be written as style="background-color:red;"

Adding a Caption

- The <caption> tag defines a table caption.
- The <caption> tag must be inserted immediately after the <table> tag.
- By default, a table caption will be center-aligned above a table. However, the CSS properties text-align and caption-side can be used to align and place the caption.
- Example:

```
<table>
  <caption>Monthly savings</caption>
  <tr>
    <th>Month</th>
    <th>Savings</th>
  </tr>
  <tr>
    <td>January</td>
    <td>$100</td>
  </tr>
</table>
```

Setting the Width

- The width attribute specifies the width of a table or the width of a table cell.
- The width can be set either as an absolute value in pixels, or as in percentage (%).
- If the width attribute is not set, it will take up the space of longest single word in each cell.
- Example
 - `<table border=1 width=100>` (table width in pixels)
 - `<table border=1 width=100%>` (table width in %)
- We can also set the width of a table cell using width attribute.
 - `<td width=30%>`

Adding a Border

- The HTML Table Border attribute sets the width of Border around the table and table cells. We can set the Border attributes in pixels.
- Example:
`<table border="2">`
 - In the above HTML code we set the Border attribute to 2, this applies a border to each cell, and to the table as a whole.
 - But whatever values we set in the Border attribute, only the size of the border around the table will change, the border of the table cells remain 1 pixel width.
- HTML Table Bordercolor
 - The bordercolor attribute will change the color of the border around our table. You can directly type the color names or we can use hexadecimal codes of colors to boardercolor attribute.
 - Example:
`<table border=5 Bordercolor=red>`
 - We can set additional two border color attributes to HTML Table, they are bordercolorlight and bordercolordark attributes. It will give you a 3D effect to your HTML Table.
`<table border=5 bordercolorlight=green bordercolordark=blue>`
- By default, there is no border around HTML Table and Table Cells. If we want to hide (No Border) Border permanently, we can set this property to either 0 or -1 .

Spacing Within a Cells

- Cellspacing:

- The Cellspacing attribute places space, in pixels, around each cell in the table.

```
<table border="1" cellspacing="5" Bordercolor="red" >
```

- The above code will Set the space between the cells to 5 pixels.
- If we want no spaces at all, we should set Cellspacing="0", otherwise the default is Cellspacing="1" will set, even if we don't mention Cellspacing.
- The cellspacing is applied both vertically and horizontally. The cell spacing is uniform for the entire table. Individual cell spacing between each row or column cannot be specified.



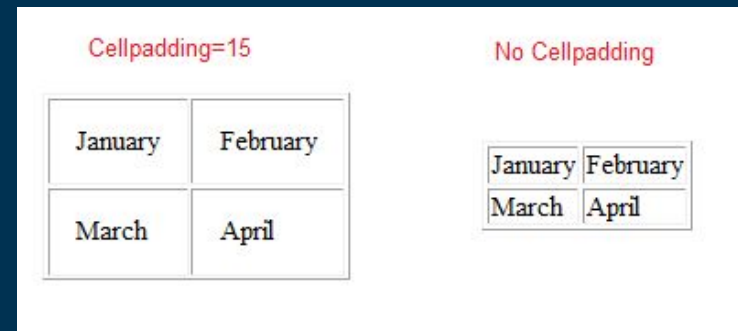
January	February
March	April

- Cellpadding:

- CellPadding attribute is used to control the spacing between the contents of a Cell and the Cell's border.

```
<table border="1" cellpadding="15">
```

- The above code set the space between the Cell border and the Cell content to 15 pixels.



Cellpadding=15		No Cellpadding	
January	February	January	February
March	April	March	April

Spanning Multiple Rows or Columns

- There are two ways for Spanning Multiple Rows or Columns in a HTML Table:
 1. `colspan`
 2. `rowspan`
- Sometimes it makes sense for a cell to span multiple columns or multiple rows. This might be used for a header cell that titles a group of columns, or a side-bar that groups rows of entries.
- These are attributes of
 - `<th>` — table header — A header cell in a `<table>`.
 - `<td>` — table data — A data cell in a `<table>`.
- Both `colspan="value"` and `rowspan="value"` are attributes of the two table-cell elements, `<th>` and `<td>`. They provide the same functionality as “merge cell” in spreadsheet programs like Excel.
- The value of either attribute must be a positive integer (a whole number). The value specifies the number of columns or rows that the cell fills.

Spanning Multiple Rows or Columns cont...

- colspan
 - Allows a single table cell to span the width of more than one cell or column.

- Example

<table>

<caption>Life Expectancy By Current Age

</caption>

<tr>

<th colspan="2">65</th>

<th colspan="2">40</th>

<th colspan="2">20</th>

</tr>

<tr>

<th>Men</th>

<th>Women</th>

<th>Men</th>

<th>Women</th>

<th>Men</th>

Spanning Multiple Rows or Columns cont...

- rowspan
 - Allows a single table cell to span the height of more than one cell or row.

- Example

```
<table>
```

```
  <caption>Favorite and Least Favorite Things</caption>
```

```
  <tr>
```

```
    <th></th><th></th>
```

```
    <th>Bob</th>
```

```
    <th>Alice</th>
```

```
  </tr>
```

```
  <tr>
```

```
    <th rowspan="2">Favorite</th>
```

```
    <th>Color</th>
```

```
    <td>Blue</td>
```

```
    <td>Purple</td>
```

```
  </tr>
```

```
  <tr>
```

```
    <th>Flavor</th>
```

```
    <td>Banana</td>
```

```
    <td>Chocolate</td>
```

Elements that can be Placed in a Table

- Note: The <td> elements are the data containers of the table.
- They can contain all sorts of HTML elements; text, images, lists, other tables, etc.
- Example: order list inside td element

```
<table style="width:100%">
```

```
<tr>
```

```
<th>Firstname</th>
```

```
<th>Lastname</th>
```

```
<th>Age</th>
```

```
</tr>
```

```
<tr>
```

```
<td>
```

```
<ol start="5">
```

```
<li>Coffee</li>
```

```
<li>Milk</li>
```

All Table Related Elements

Element	What it is
<table>	The table itself
<caption>	The caption for the table. Like a figcaption to a figure.
<thead>	The table header
<tbody>	The table body
<tfoot>	The table footer
<tr>	A table row
<th>	A table cell that is a header
<td>	A table cell that is data
<col>	A column (a no-content element)
<colgroup>	A group of columns

Column Properties

- **colgroup**

- The `<colgroup>` tag specifies a group of one or more columns in a table for formatting.
- The `<colgroup>` tag is useful for applying styles to entire columns, instead of repeating the styles for each cell, for each row.
- Note: The `<colgroup>` tag must be a child of a `<table>` element, after any `<caption>` elements and before any `<thead>`, `<tbody>`, `<tfoot>`, and `<tr>` elements.
- Tip: To define different properties to a column within a `<colgroup>`, use the `<col>` tag within the `<colgroup>` tag.

- **col**

- The `<col>` tag specifies column properties for each column within a `<colgroup>` element.
- The `<col>` tag is useful for applying styles to entire columns, instead of repeating the styles for each cell, for each row.

Column Properties cont...

Example:

```
<table>
  <caption>Superheros and sidekicks
</caption>
  <colgroup>
    <col>
    <col span="2" class="batman">
    <col span="2" class="flash">
  </colgroup>
  <tr>
    <td> </td>
    <th scope="col">Batman</th>
    <th scope="col">Robin</th>
```

Table as a Design Tool

- Tables are useful for laying out text and images on in Web page.
- Big tables are a nightmare for users trying to view them on the small screens of mobile device screen. But if you are careful, you can use tables to good effect in some situations.
- By default, all Web pages are designed with flexibility in mind. Flexibility can be defined as a Web page's ability to resize and adapt to the available resolution, monitor and window sizes. Such an approach has both advantages and disadvantages.

- Advantages:

Tables in HTML have several advantages that you are probably used to taking advantage of in desktop Web design:

- The HTML table model allows authors to arrange data -- text, preformatted text, images, links, forms, form fields, other tables, etc. -- into rows and columns of cells.
- Columns can resize automatically, so the table always fills different screen widths.
- Cell borders and background colors can enhance readability.

- Disadvantages of tables

Tables also have distinct disadvantages on mobile devices:

- They are less flexible, and the contents cannot move from their assigned positions or overlap.
- Images must be sliced up to fit in a cell.
- Page size is increased vs. the same content displayed without a table.
- Tables are slower to load.
- Tables have fewer styling controls.

1.3 Html basics: List Tags

- HTML lists allow web developers to group a set of related items in lists.
- Three ways for specifying lists of information. All lists must contain one or more list elements. Lists may contain:
 - `<ul>` – An unordered list. This will list items using plain bullets.
 - `<ol>` – An ordered list. This will use different schemes of numbers to list your items.
 - `<dl>` – A description list. This arranges your items in the same way as they are arranged in a dictionary.
- Unordered List
 - An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag.
 - The list items will be marked with bullets (small black circles) by default:
 - Example:

```
<ul>  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ul>
```
- Ordered HTML List
 - An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag.
 - The list items will be marked with numbers by default
 - Example:

```
<ol>  
  <li>Coffee</li>  
  <li>Tea</li>  
  <li>Milk</li>  
</ol>
```

HTML Description Lists

- A description list is a list of terms, with a description of each term.
- The `<dl>` tag defines the description list, the `<dt>` tag defines the term (name), and the `<dd>` tag describes each term:

- Example:

```
<dl>
  <dt>Coffee</dt>
  <dd>- black hot drink</dd>
  <dt>Milk</dt>
  <dd>- white cold drink</dd>
</dl>
```

- HTML List Tags

Tag Description

- | | |
|-------------------------|--|
| <code></code> | Defines an unordered list |
| <code></code> | Defines an ordered list |
| <code></code> | Defines a list item |
| <code><dl></code> | Defines a description list |
| <code><dt></code> | Defines a term in a description list |
| <code><dd></code> | Describes the term in a description list |

Lists Attribute

Attribute:

Attribute	Value	Description
<u>value</u>	<i>number</i>	Only for lists. Specifies the start value of a list item. The following list items will increment from that number
type	Square, disc, circle	for tag to specify the type of bullet you like. By default, it is a disc.
	1, I, i, A, a	Resp. for tag By default 1; Case Numerals, Upper-Case Numerals, Lower-Case Numerals, Upper-Case Letters, Lower-Case Letters

```
<ol>
  <li value="100">Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
  <li>Water</li>
  <li>Juice</li>
  <li>Beer</li>
</ol>
```

1.3 Html basics: semantic HTML5 elements

- A semantic element clearly describes its meaning to both the browser and the developer.
 - Examples of semantic elements: `<img>`, `<table>`, and `<article>` - Clearly defines its content.
 - Examples of non-semantic elements: `<div>` and `<span>` - Tells nothing about its content.
- Key characteristics and benefits of Semantic HTML5:
 - Clarity and Readability: Semantic tags make the code more understandable and self-documenting, as the purpose of each section is immediately clear from its tag name.
 - Accessibility: Semantic elements greatly improve accessibility for users with disabilities, particularly those using screen readers and other assistive technologies. These tools can interpret the page structure more effectively, enhancing navigation and content comprehension.
 - Search Engine Optimization (SEO): Search engines can better understand and index the content of a semantically structured webpage. This can lead to improved content indexing, better keyword relevance assessment, and potentially higher search engine rankings.
 - Maintainability and Collaboration: A well-structured semantic HTML document is easier to maintain and collaborate on, as developers can quickly grasp the layout and content organization.
 - Improved Responsiveness and Adaptability: Semantic HTML facilitates better responsive design implementation and browser compatibility, ensuring content adapts well across various devices and platforms.

1.3 Html basics: semantic HTML5 elements (cont...)

- Examples of common semantic HTML5 elements:
 - `<header>`: Represents introductory content or a set of navigational links, typically found at the top of a page or section.
 - `<nav>`: Defines a set of navigation links.
 - `<main>`: Represents the dominant content of the `<body>` of a document.
 - `<article>`: Specifies independent, self-contained content, such as a blog post or news article.
 - `<section>`: Defines a thematic grouping of content.
 - `<aside>`: Represents content that is tangentially related to the content around it, often presented as a sidebar.
 - `<footer>`: Represents the footer of a document or section, typically containing copyright information, contact details, or related links.
 - `<figure>` and `<figcaption>`: Used to embed self-contained content (like images, diagrams, or code snippets) with an optional caption.
 - `<time>`: Represents a specific point in time or a time range.